# Agent-Controlled RAG for Reliable Document Question Answering

Thanneeru Venkateswarlu, Amirisetti Mahesh Gopi, Marre Hemanth Kumar, Gokina Charan, and P. Syamala Rao*

*Department of Information Technology, Sagi Ramakrishnam Raju Engineering College (Autonomous),*
*Bhimavaram – 534202, Andhra Pradesh, India*
venkateswarluthanneeru123@gmail.com, maheshgopi023@gmail.com, marrrehemanthkumar@gmail.com, gokinacharanchowdary@gmail.com
*Corresponding author: Peketi.shyam@gmail.com

**Abstract.** There are two common failures of RAG-based question answering systems: depth to retrieval is fixed, irrespective of the complexity of the query, and responses generated are not compared to the evidence found. This paper introduces AC-RAG, a multi-agent system, in which distinct modules are dealing with query comprehension, query-planning, context-refinement, response-generation, and introspection. The system varies retrieval depth depending on the complexity of queries instead of making use of a fixed strategy approach. Once generated, a critic agent rates the answer on five dimensions and retries the answer in case the quality threshold is not reached. RAGAS We tested the system on document QA tasks using RAGAS. The findings indicate better answer faithfulness, relevance to context and accuracy in responses compared to a baseline RAG system. No finesetting of the models. The structure operates with text, table, and figure documents.

**Keywords:** *Retrieval-Augmented Generation (RAG), Document Question Answering, Adaptive Retrieval, Self-Reflection, Answer Faithfulness, Multi-Agent Framework, AC-RAG, RAGAS*

## 1 Introduction

It is now common to use large language models to answer questions on document collections. Retrieval-Augmented Generation (RAG) assists in that external evidence is drawn in during generation and this makes the model less dependent on the information that the model already possesses. The limitations of standard RAG pipelines are however not always easy to overcome without reorganizing the method. The first obvious restriction is that most systems recall as many passages as possible per query. An easy descriptive question requires much less context than an analytical one that involves many steps. In addition to retrieval, not all systems verify that the generated answer is supported by what was retrieved. Unsupported statements can go through undetected. Previous literature has primarily been working on enhancing the quality of retrieval by using improved embedding models and reranking. This is helpful, but the generation phase is relatively

unregulated. The entire system still might generate responses grounded in the available evidence. A design with agents provides a means to address the two issues. Everything is assigned a particular task, so it is possible to modify retrieval per query and introduce a validation step prior to providing the response. AC-RAG is a multi-agent architecture of document question answering, as is presented in this paper. The system encompasses query understanding components, retrieval planning components, evidence validation components, context refinement components, answer generation components, and self-reflection components. The degree of retrieval depth is modified about the complexity of queries and a critic module compares each response to the retrieved content and only when the response matches the content is it delivered. This is aimed at enhancing the reliability of answers based on the varying document types without necessarily varying the underlying model.

## 2 Literature Review

The old techniques of question answering relied on key-word matching and retrieval by rule. These did not do well with unstructured text and queries which had to be understood in context. This was enhanced with the introduction of neural language models which helped to better understand the queries and the documents semantically. RAG proposed by Lewis et al. [1] is a method that integrates document retrieval and sequence generation to enhance the factual grounding. Retrieval was further improved by dense Passage Retrieval (DPR) [2] which learned dense representations of question-passage pairs, and Fusion-in-Decoder (FiD) [3] which learned to combine multiple passages retrieved at once during decoding. The combination of these works made retrieval one of the primary components of contemporary question answering. These developments notwithstanding, there are still some issues. Most of the RAG systems find a fixed number of passages to each query, no matter how complicated the question is. It may result in partial evidence of complicated queries or irrelevant data of straightforward ones. There is also a problem of hallucination. The research like Maynez et al. [4] indicates that outputs generated are not necessarily consistent with the original information. Evaluation FactCC like FactCC [5] and similar work [6] are tools that measure factual consistency; however, they are primarily evaluation tools. They are not inbuilt in the process of generation itself. Recent research has examined agent-based reasoning to enhance system behaviour. ReAct [8] integrates steps of reasoning with the use of a tool, whereas Reflexion [9] illustrates that the performance can be enhanced through the iterative feedback provided in a series of attempts. These approaches enhance reasoning, and none of them was crafted with the purpose of document-based QA.

More recent works have looked at agentic RAG architectures. The difference between retrieval-oriented and generation-oriented gains is pointed out in the surveys like Gao et al. [10]. Modular agent-based systems like the one suggested by Panda [11] offer relaxed pipeline architecture but do not emphasize much on verification processes of generated responses. These are the areas of retrieval and reasoning, which are not considered in detail, even with earlier work, which has enhanced them significantly. Adaptive retrieval, structured validation, and controlled generation still have a limited presence in a single framework. AC-RAG is designed and constructed to place these components within a single pipeline.

## 3 Proposed Methodology

AC-RAG is a document-based question answering multi-agent pipeline. Instead of applying a single model to all, it divides the work into several distinct steps with a particular role assigned to each one. The resulting modular design allows controlled recall, organization of validation and refinement of the produced responses.

### *3.1 System Overview*

This system is implemented by using a series of steps and in each step, a query is processed because of the step that precedes it. The pipeline consists of some retry mechanisms that have predetermined limits so that it does not run forever. In case the response that has been generated fails to comply with pre-set criteria, the system restarts at a previous stage with modified inputs.

The general one is query understanding, then retrieval planning, document retrieval, evidence validation, context refinement, answer generation, self-reflection.

**Problem Formulation.** Given a collection of documents D and a query q, the goal is to produce an answer a in a way that satisfies the following:

$$a = \mathrm{argmax}\, P(a \mid q, R(q, D))$$

or R(q, D) is the evidence that was retrieved. Moreover, the system has a faithfulness restrictive element:

$$Faith(a, R(q, D)) \geq \tau$$

where τ is a minimum threshold that is imposed on numerous evaluation dimensions. This limitation is guaranteed so that the answer created is based on what is actually being retrieved.

### *3.2 Query Understanding*

The first step examines the input request to identify its purpose and level of sophistication. The query is rephrased to a form accessible to search and categorized as either detailed or brief depending on the amount of detail required. When dealing with more complicated queries, the query will be broken down into smaller sub-queries that are independent of each other. This will minimize ambiguity and better the relevance of the retrieved material to what the query really requires.

### *3.3 Retrieval Planning*

As opposed to the conventional RAG systems, which adopt a predetermined retrieval strategy, the retrieval strategy at this stage is the one that identifies the quantity of passages to be retrieved depending on the complexity of the query. Complex queries are given more depth of retrieval and less complex queries utilize fewer passages so as not to add noise. The size of retrieval, k is dynamically chosen between 4 and 12. The planner can also choose the appropriate content modalities i.e. text, tables or figures based on the query nature.

### *3.4 Document Retrieval*

Strictly speaking, the documents are broken down into chunks and stored in different, modality-based, vector stores. The retrieval is done on Maximum Marginal Relevance (MMR) which balances both similarity and diversity to minimize redundancy in the retrieved set. The sub-queries are processed individually, and the passages retrieved. The latter is useful in cases where the answer requires content distributed in various sections of the document.

### *3.5 Evidence Validation*

The passages that are retrieved are judged separately in terms of relevancy. A score is given to each passage according to its similarity to the query, and any with a score less than 0.65 is eliminated. When the rest set is not enough, the system will revert to retrieval planning stage with feedback of adjusting the retrieval parameters. This prevents poor or irrelevant passages to get to the stage of generation.

### *3.6 Context Refinement*

Passages which have been validated are further edited to enhance coherence and minimise redundancy. Duplicative or redundant text is consolidated and poorly informative material is eliminated. The rest of the passages are ranked and joined into one context window. The output is a cleaner and more concentrated input in the generation phase.

### *3.7 Answer Generation*

Response is elicited with the help of a language model that is conditioned on the refined context. The model is orientated to generate solutions that do not go out of the evidence presented and do not create unsubstantiated information. Everything in the response must be traced to a particular passage that was retrieved.

### *3.8 Self-Reflection*

In the last phase, the response that is generated is tested on five dimensions faithfulness, completeness, table accuracy, figure accuracy, and conciseness. The scores of the dimensions fall on a scale of 1 to 5. It is only when all the scores reach or surpass a threshold of 4 that the response is accepted. In the event of failure of the response because of content related problems, the system goes back to the query understanding phase with new inputs. In case of problems with the format, the regeneration is made with the same context. The number of retries in both retry controls is limited to three, which means that the system will not run forever when the initial outputs are bad.

## 4 System Architecture

AC-RAG is constructed in the form of a modular pipeline with every building taking the output of the previous building. The control of agent coordination, shared state and conditional routing are handled with LangGraph. A common state object is passed down the full pipeline carrying query, passages already retrieved, scores of validations and the answer generated at every step. The system has three query-based entry routes. Language model takes care of conversational queries, web-based queries are redirected to an external search engine and document related queries are addressed by the entire multi-agent pipeline. This eliminates unnecessary running of retrieval over queries that do not require it.

### *4.1 Component Overview*

The architecture has seven major components that are organized in a series:

- Router: Categorizes the query as somebody comes into the router and forwards it to the right processing route. All queries on the document are exclusively sent to the retrieval pipeline.
- Query Understanding: Parses the query to identify intent and complexity and reforms the query to be retrieved and breaks down the query into sub-queries where required.
- Retrieval Planning: Decides the depth of retrieval and modalities of the relevant content depending on the query processed.

- Document Retrieval: Maximum Marginal Relevance (MMR) is the method used to retrieve passages of candidates, which are stored in modality specific vector stores.
- Validation of Evidence: The relevance of passages retrieved is also measured and low-quality content is filtered. In case of a lack of enough evidence, the feedback is provided to amend retrieval parameters.
- Context Refinement: Removes the redundancy, filters weak passages, and forms one context window.
- Generation of Answers: Generates a response based on a language model that has been trained on the narrowed down context.
- Self-Reflection: Assesses the produced response in the context of several quality dimensions and decides on whether to accept the response, re-generate or re-initiate the pipeline.

### *4.2 System Architecture Diagram*

Fig.1 depicts the entire AC-RAG pipeline with all the agents, modality-specific stores of vectors, the feedback-reflection loop, and provided safeguards.

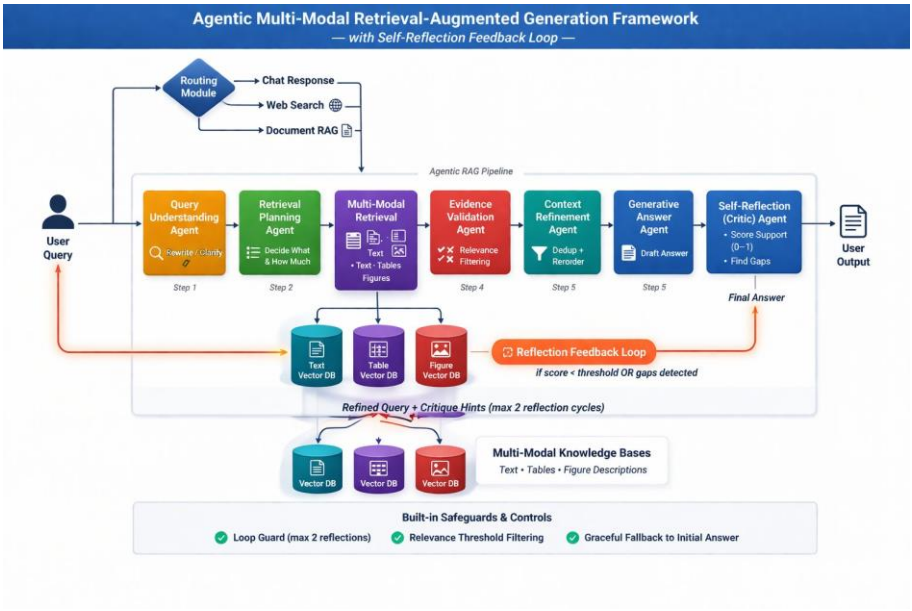


*Fig. 1. Architecture AC-RAG system: Agentic Multi-modal Retrieval- Augmented Generation Framework with Self- Reflexion Feedback Loop. The figure depicts routing module, 7 stage pipeline, multi-modal knowledge bases and loop guard controls.*

### *4.3 Comparison with Prior Systems*

The comparative analysis of AC-RAG and the representative prior systems about eight architectural dimensions is presented in Table 1.

| System | Dense Retr. | Adapt. Retr. | Multi Agent | Evid. Valid. | Ctx Refine | Self-Refl. | Feed-back | No Fine-tune |
|---|---|---|---|---|---|---|---|---|
| RAG (2020) [1] | ✓ | ✗ | ✗ | ✗ | ✗ | ✗ | ✗ | ✓ |
| DPR (2020) [2] | ✓ | ✗ | ✗ | ✗ | ✗ | ✗ | ✗ | ✗ |
| FiD (2021) [3] | ✓ | ✗ | ✗ | ✗ | ✗ | ✗ | ✗ | ✗ |
| ReAct (2023) [8] | ~ | ~ | ✗ | ✗ | ✗ | ✗ | ~ | ✓ |
| Reflexion (2023) [9] | ✗ | ✗ | ✗ | ✗ | ✗ | ✓ | ~ | ✓ |
| Toolformer (2023) | ✗ | ✗ | ✗ | ✗ | ✗ | ✗ | ✗ | ✓ |
| Agentic RAG (2025) [11] | ✓ | ~ | ~ | ✗ | ✗ | ✗ | ✗ | ✓ |
| **AC-RAG (Ours)** | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ |

*Table 1. Comparison of architecture Architectural comparison. ✓ = supported; ✗ = not supported; ~ = partially supported. There are eight criteria that AC-RAG meets concurrently.*

### *4.4 Data Flow*

Data flow in most cases will be through the pipeline directly to the next stage. A query is sent through the router and into the query understanding stage where it is rewritten and can be broken down if needed. The processed query goes to the retrieval planner and identifies retrieval parameters. The retrieval module retrieves passages in the vector stores and sends it to the evidence validation step. The passages that pass the filter are then refined into a single context window and are sent to answer generation. The self-reflection component then measures the response generated. There are also feedback paths in the system. In case of content related failures, control goes back to query understanding stage followed by corrective feedback. In the case of problems related to format regeneration is done with the same context.

### *4.5 Vector Store Design*

Preprocessing of documents is done before use of query. The documents are broken into blocks of about 600 characters in groups of 100 characters overlap. Information is stored as three different ChromaDB collections that represent text, tables, and figures. The separation of these implies that the retrieval planner will only be able to focus on the specific type of content that is relevant to a particular query. The content of the figures is analyzed as a vision-capable model that produces textual descriptions in human-readable form and then makes visual information searchable through typical textual search.

### *4.6 Retry and Termination*

The system encompasses autonomously capped restarting functions. A retrieval retry counter restricts the number of attempts to get back to the retrieval planning stage, whereas a reflection retries counter controls the number of regeneration attempts. Each of the counters is limited to three repeats. After the limit has been hit, the system delivers the best response possible. This holds the pipeline constant, and does not allow the pipeline to run forever.

## 5 Experimental Setup

### *5.1 Dataset and Query Types*

The evaluated system worked with a set of technical and academic PDF documents related to the fields of artificial intelligence and natural language processing. The reports had a heterogeneous evaluation setting with mixed content such as running text, tabular content, and embedded figures. A set of 50 question-answer pairs were selected to be evaluated. These questions were categorized into three types namely factual questions that needed a direct answer which could be found in a single passage, analytical questions that needed information found in several sections and table or figure questions which needed numerical or visual information. The distribution was made to test the system on various levels of query complexity.

### *5.2 Tools and Models*

A pipeline coordination system, the LangGraph, and a vector storage system, the ChromaDB, were used to construct the system. Understanding query and answer generation were done using GPT-4o. Self-reflection step involved the action of using o3-mini to evaluate responses. There was the generation of text descriptions of figure content using a vision-capable model. The text-embedding-3-large model was used to create embeddings.

### *5.3 Evaluation Metrics*

The RAGAS framework [12] was used to evaluate the system by offering automated measures to be specifically used with retrieval-augmented generation systems. These were the following metrics:

- Answer Faithfulness - gauges the support of the answer that has been generated by the retrieved context.
- Context Relevance — evaluates the relevance of passages that are retrieved to the query.

- Relevancy of the Answer — is the measure of whether the answer is a response to the question.
- Context Recall — the amount of the needed information was encoded into a retrieved set.
- Hallucination Rate (inversed as reduction) This is the proportion of the responses which do not include unsupported claims.

The accuracy of responses was also determined with comparison of generated answers with manually prepared reference answers.

### *5.4 Baseline Comparison*

AC-RAG was contrasted to a typical RAG system [1] that operates on the same language model and document set. The baseline involved a predetermined depth of retrieving five passages on all queries and lacked of evidence validation and self-reflection. This guarantees that any improvements that are observed can be attributed to architectural design as opposed to variations in the underlying model.

### *5.5 Parameter Settings*

The major parameters involved in the experiments are as identified in Table 2.

| Parameter | Value |
|---|---|
| Chunk size | 600 characters |
| Chunk overlap | 100 characters |
| Retrieval range (k) | 4 to 12 |
| Evidence threshold | 0.65 |
| Self-reflection threshold | 4 out of 5 |
| Max retrieval retries | 3 |
| Max reflection retries | 3 |
| Embedding model | text-embedding-3-large |
| Generation model | GPT-4o |
| Critic model | o3-mini |

*Table 2. Parameters of the AC-RAG experiment.*

This architecture provides a balanced and a managed testing of the suggested system with various query types and document modalities.

## 6 Results and Discussion

### *6.1 Overall Results*

Table 3 lists results on RAGAS evaluation [12] of AC-RAG and the baseline RAG system [1] on the curated 50 question-answer pairs in the set of 50 question-answer pairs outlined in the paper in Section 5. The two systems had been tested on the same document set with the same evaluate framework and this gave a level of fairness in the comparison. It does not involve a directly numerical comparison with other systems like ReAct [8], Reflexion [9] and Agentic RAG [11], which have been tested on different datasets and show different measures. A qualitative comparison of architectural capabilities across these systems is given in table 1.

| System | Faithfulness (AF ↑) | Halluc. Red. (HR ↑) | Ctx Relevance (CR ↑) | Resp. Acc. (RA ↑) |
|---|---|---|---|---|
| Baseline RAG [1] | 0.61 | 0.58 | 0.64 | 0.63 |
| **AC-RAG (Ours)** | **0.88** | **0.85** | **0.86** | **0.87** |

*Table 3. Baseline RAG and AC-RAG results on Baseline RAG evaluation and the same document QA dataset (0 1 scale, where higher is better). These two systems have the same language model backbone.*

AC-RAG had a score higher than the baseline on all four measures. The highest change was in answer faithfulness which changed to 0.88 and 0.61 respectively. The same trend is observed in the reduction of hallucinations that go between 0.58 and 0.85. This is an indication that the stage of evidence validation and self-reflection do have a quantifiable influence on the quality of responses.

### *6.2 Discussion*

#### Answer Faithfulness and Hallucination Reduction

The 0.61 to 0.88 faithfulness enhancement has a connection that is perhaps related to two aspects. The first stage is the evidence validation phase which eliminates passages that are of low significance prior to generation. Second, the self-reflection phase compares response with the content that is retrieved and then the response is finalised. In the baseline system, the passages that are retrieved are sent straight to the generator without any filtering and this has the ability of introducing noise and may result in responses not being grounded in the source.

#### Context Relevance

The context relevance increased by 0.64 to 0.86. This is in line with adaptive retrieval strategy where the number of passages retrieved is modified according to the complexity of a query. Simple queries find fewer passages and minimise irrelevant passages. In complicated queries, the expanded range of retrieval enables more information that is of interest to be retrieved.

### Answer Relevancy

Relevancy on answers changed to 0.86 (it was 0.67 before). This is probably facilitated by the query understanding phase that rewrites and breaks down the input and then is retrieved and thus enhances the match between the query and the retrieved information.

### Comparison with Other Approaches

It is not possible to numerically compare it to systems like ReAct [8], Reflexion [9], and Agentic RAG [11] since the latter have not been tested on the same data and metrics. Nevertheless, a comparative analysis of the architectural aspects in these systems has been presented in Table 1 in a qualitative manner. The only system in this comparison that can be used to support adaptative retrieval, evidence validation, self-reflection and feedback-based retry at the same time is AC-RAG. This architectural wholesomeness is the key factor of the performance improvements compared to the baseline.

## *6.3 Ablation Study*

Table 4 is a quantification of the contribution of each AC-RAG component, by subtracting them at runtime.

| Variant | AF ↑ | HR ↑ | CR ↑ | RA ↑ |
|---|---|---|---|---|
| **Full AC-RAG** | **0.88** | **0.85** | **0.86** | **0.87** |
| w/o Self-Reflection | 0.73 | 0.70 | 0.86 | 0.74 |
| w/o Evidence Validation | 0.75 | 0.72 | 0.72 | 0.76 |
| w/o Adaptive Retrieval (fixed k=5) | 0.80 | 0.77 | 0.79 | 0.81 |
| w/o Multi-Modal Retrieval | 0.79 | 0.76 | 0.71 | 0.78 |
| w/o Context Refinement | 0.82 | 0.78 | 0.86 | 0.83 |
| w/o Router (always RAG) | 0.88 | 0.85 | 0.86 | 0.86 |

*Table 4. Ablation study results. The components that get removed by each row are one. The upper bound is called Top row (Full AC-RAG). Elimination of self-reflection will lead to the greatest one-unit decrease in faithfulness (−0.15).*

Eliminating self-reflection results in the greatest percentage decrease in answer faithfulness (0.88 -0.73, -0.15) which validates it as the most important element. The elimination of evidence validation leads to the second-largest decrease in faithfulness (→ 0.75) and context relevance (→ 0.72). The faithfulness is minimized to 0.80 by deleting adaptive retrieval. The elimination of multi-modal retrieval influences the context relevance significantly (→ 0.71). The least

impacting one is context refinement removal (→ 0.82 AF). As it is expected, the router ablation is almost neutral on faithfulness metrics.

### *6.4 Limitations*

The findings are positive, yet there are certain weaknesses that should be mentioned. To begin with, the assessment was performed based on a rather small sample of 50 pairs of questions and answers. The findings might not be applicable to bigger or more diverse document sets. Second, the RAGAS measures are automated and are based on model-based scoring, which is not always appropriate to the quality of responses. The extra steps also add to the response time and this might be a limitation in applications that are speed sensitive. Lastly, self-reflection stage applies thresholds by all query types and this may not be applicable to all document types.

## 7 Conclusion and Future Work

### *7.1 Conclusion*

AC-RAG is a multi-agent pipeline of document-based query answering that was presented in this paper. It was developed to tackle two issues on standard RAG: fixed retrieval depth, and no answer verification. The suggested method breaks the pipeline into stages that are highly specialised such as query understanding, retrieval planning, evidence validation, context refinement, answer generation, and self-reflection. Measures based on RAGAS indicated an increase in several measures relative to a standard RAG baseline. Specifically, the increase in the answer faithfulness, and loss of supporters give reason to believe that a combination of adaptive retrieval and structured validation results in more trustworthy results. All these gains did not involve modifications to the underlying language model. Although such improvements have been made, there are still several limitations. The analysis has been done based on a limited dataset, and this might influence generalisability. Self-reflection mechanism also uses predetermined thresholds, and the multi-stage pipeline has more latency than simpler methods. The findings indicate that retrieval control plus answer validation is a feasible method of enhancing reliability in document QA especially where accuracy is required.

### *7.2 Future Work*

Based on these observations, below are some directions which can be identified in future work.

Larger-scale evaluation. Assessing the system based on more significant benchmarks and domain-specific datasets would give a more detailed analysis of the performance under different conditions.

Dynamic thresholding. Future research may investigate an adaptive adjustment of the validation thresholds in accordance with the nature of queries or documents.

latency optimisation technique. Response time could be improved by reducing the model calls or running the independent stages in parallel.

Additional document formats. The system should be expanded to support other types of documents like scanned documents, spreadsheets, or presentation slides to increase the scope of application.

These guidelines give a route of enhancing the strength as well as scalability of the proposed system.

### *Declaration*

The authors of this paper created the architecture diagram that this paper presents. We have received a license to use the images/data provided in the paper by competent persons. We shall be entirely liable in the event of any dispute in future.

## References


1. Lewis P, Perez E, Piktus A, Petroni F, Karpukhin V, Goyal N, Küttler H, Lewis M, Yih W, Rocktäschel T, Riedel S, Kiela D (2020) Retrieval-augmented generation for knowledge-intensive NLP tasks. Adv Neural Inf Process Syst 33:9459–9474 https://proceedings.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html
2. Karpukhin V, Oguz B, Min S, Lewis P, Wu L, Edunov S, Chen D, Yih W (2020) Dense passage retrieval for open-domain question answering. In: Proc EMNLP 2020, pp 6769–6781 doi:10.18653/v1/2020.emnlp-main.550. https://aclanthology.org/2020.emnlp-main.550/
3. Izacard G, Grave E (2021) Leveraging passage retrieval with generative models for open domain question answering. In: Proc EACL 2021, pp 874–880 doi:10.18653/v1/2021.eacl-main.74. https://aclanthology.org/2021.eacl-main.74/
4. Maynez J, Narayan S, Bohnet B, McDonald R (2020) On faithfulness and factuality in abstractive summarization. In: Proc ACL 2020, pp 1906–1919 doi:10.18653/v1/2020.acl-main.173. https://aclanthology.org/2020.acl-main.173/
5. Kryscinski W, McCann B, Xiong C, Socher R (2020) Evaluating the factual consistency of abstractive text summarization. In: Proc EMNLP 2020, pp 9332–9346 doi:10.18653/v1/2020.emnlp-main.750. https://aclanthology.org/2020.emnlp-main.750/
6. Pagnoni A, Balachandran V, Tsvetkov Y (2021) Understanding factuality in abstractive summarization with FRANK: a benchmark for factuality metrics. In: Proc NAACL-HLT

2021, pp 4812–4829 doi:10.18653/v1/2021.naacl-main.383. https://aclanthology.org/2021.naacl-main.383/

7. Wang X, Wei J, Schuurmans D, Le QV, Chi EH, Narang S, Chowdhery A, Zhou D (2023) Self-consistency improves chain of thought reasoning in language models. In: Proc ICLR 2023 https://openreview.net/forum?id=1PL1NIMMrw

8. Yao S, Zhao J, Yu D, Du N, Shafran I, Narasimhan K, Cao Y (2023) ReAct: synergizing reasoning and acting in language models. In: Proc ICLR 2023 https://openreview.net/forum?id=WE_vluYUL-X

9. Shinn N, Cassano F, Gopinath A, Narasimhan K, Yao S (2023) Reflexion: language agents with verbal reinforcement learning. Adv Neural Inf Process Syst 36:8634–8652 https://proceedings.neurips.cc/paper_files/paper/2023/hash/1b44b878bb782e6954cd888628510e90-Abstract-Conference.html

10. Gao Y, Xiong Y, Gao X, Jia K, Pan J, Bi Y, Dai Y, Sun J, Wang M, Wang H (2023) Retrieval-augmented generation for large language models: a survey. arXiv:2312.10997 doi:10.48550/arXiv.2312.10997. https://arxiv.org/abs/2312.10997

11. Panda M (2025) Agentic RAG: redefining retrieval-augmented generation for adaptive intelligence. Int Res J Eng Technol (IRJET) 12(1):731–738 https://www.irjet.net/archives/V12/i1/IRJET-V12I1112.pdf

12. Es S, James J, Anke LE, Schockaert S (2024) RAGAS: automated evaluation of retrieval augmented generation. In: Proc EACL 2024 Syst Demonstrations, pp 150–158 doi:10.18653/v1/2024.eacl-demo.16. https://aclanthology.org/2024.eacl-demo.16/